

第2章 算法分析

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解为什么"程序能跑"远远不够——**复杂度**决定程序在数据变大时能不能跑完
- 掌握 O 、 Ω 、 Θ 、 o 四种记号的准确定义与直觉,能说出它们的区别
- 能背诵增长阶排序:常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数
- 会计算顺序/循环/递归代码的运行时间,能把循环翻译成求和记号 Σ
- 亲手实现**最大子序列和**问题的四种算法,体会 $O(n^3)$ 到 $O(n)$ 的天壤之别
- 能说明二分查找、快速幂、欧几里得算法为什么都是 $O(\log n)$
- 学完能独立用 `time.perf_counter` (C 中对应 `clock()`) 实测验证自己的复杂度分析

💡 从 C 到 Python:本章主题如何迁移

本章是全书唯一"几乎不写新数据结构"的一章,但它是**全书的通用语言**:第 3 章起,每章都以"这个操作 $O(\text{多少})$ "作为结论。算法分析的直觉一句话:**复杂度是数据规模变大时,运行时间的增长方式**——不是秒数,不是毫秒,而是"规模翻倍,时间大约乘以几"。

在 C 里,你也许用过 `clock()` 计时:包含 `time.h`,前后各取一次,除以 `CLOCKS_PER_SEC`。Python 的 `time.perf_counter()` 思想相同但更省事:单调时钟、纳秒级分辨率、一行调用,不受系统修改时间的影响。本章全部计时实验都用它。

Python 还有一个 C 没有的独特优势:**整数任意精度,永不溢出**。做 §2.7 的实测实验时,你可以放心把数组元素取得很大、把 n 推到 10^7 ,而 C/C++ 的 `int` 在累加中可能悄悄溢出变负数。代价是 Python 每条语句都是解释执行、每个整数都是堆上对象——同样 $O(n)$ 的循环,Python 通常比 C++ 慢 30-100 倍。但请记住本章的核心结论:**增长阶是语言无关的**,Python 的 $O(n \log n)$ 在大数据上照样碾压 C++ 的 $O(n^2)$ 。

本章路线:先建立记号体系(§2.1-§2.3),再学计算规则(§2.4),然后在一个真实问题上亲手把 $O(n^3)$ 一路优化到 $O(n)$ (§2.5),最后用计时实验验证理论(§2.6-§2.7)。

📦 前置知识自查

数学前置:指数与对数的基本性质(第1章)、求和记号 Σ 、递归四法则与递推式入门(第1章)。

C 语言前置:单层/嵌套 `for` 循环、函数定义与调用、数组下标遍历; `clock()` 计时(可选,会更好)。

依赖的前面章节:第1章绪论(递归思想、 $\log$ 与指数的运算性质)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §2.1–§2.3, 背下增长阶排序与常见函数值表
2. **第二阶段(1.5 小时)**:读 §2.4–§2.5, 亲手推导最大子序列和四种算法的复杂度, 并在 REPL 里运行验证输出正确
3. **第三阶段(1.5 小时)**:读 §2.6–§2.7, 运行计时程序, 观察“n 翻倍、时间乘几”与理论预测是否吻合
4. **第四阶段(实验, 1 小时)**:完成章末实验: 四种算法的增长阶实测报告
5. **验收标准**:能随口说出“二分查找 $O(\log n)$ ”“快排平均 $O(n \log n)$ ”并解释理由

本章知识导图

- §2.1 为什么要分析算法:从“能跑”到“跑得完”(规模、计时工具、C 的 `clock()` 与 `perf_counter` 对照)
- §2.2 大 O 、 Ω 、 Θ 、小 o 定义、直觉、常见误用
- §2.3 增长阶排序与常见函数:七个等级、数值表
- §2.4 运行时间计算规则:顺序/循环/递归、求和记号
- §2.5 最大子序列和:四种算法, $O(n^3) \rightarrow O(n)$ 的递进
- §2.6 对数级复杂度的三种来源:二分查找、快速幂、欧几里得算法
- §2.7 实体验证:计时方法、n 翻倍实验、常数因子的意义

§2.1 为什么要分析算法

2.1.1 规模决定生死

同一个问题, 算法不同, 命运不同。假设处理一个元素耗时 1 微秒, 那么:

问题规模 n	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10^3	≈0.01 秒	1 秒	16.7 分钟	约 10^{286} 年
10^4	≈0.13 秒	100 秒	11.6 天	超出宇宙年龄
10^5	≈1.7 秒	2.8 小时	31.7 年	—
10^6	≈20 秒	11.6 天	3.2 万年	—

结论: 指数算法在任何现实规模上都不存在; 平方算法在十万级数据上就要等几小时。这就是为什么必须先学会“估计算法代价”——否则优化方向可能完全错误(去优化一个 $O(n)$ 的循环, 却对 $O(n^2)$ 的嵌套循环视而不见)。

2.1.2 从 C 的 `clock()` 到 Python 的 `perf_counter`

C 的计时套路你也许写过:

C 的写法	Python 的写法
<code>#include <time.h></code>	<code>import time</code>
<code>clock_t t0 = clock();</code>	<code>t0 = time.perf_counter()</code>
<code>double s = (double)(clock() - t0)/CLOCKS_PER_SEC;</code>	<code>elapsed = time.perf_counter() - t0</code>
精度毫秒级,测 CPU 时间	纳秒级分辨率,测墙上时间,单调不受系统改时间影响

示例 2-1 Python 计时框架(本章所有实验都用它)

```
import time

t0 = time.perf_counter()
# ---- 被测代码 ----
s = 0
for i in range(1_000_000):
    s += i
# -----
elapsed = time.perf_counter() - t0
print(f"耗时: {elapsed * 1000:.3f} ms, 结果 = {s}")
```

★ 重点:计时不是分析的全部

计时是验证分析的工具,不是分析本身。同一程序换个机器、换个负载,时间可能差几倍——但“n 翻倍、时间约乘 4”这个比值规律几乎不变。复杂度分析研究的正是这个机器无关的规律。

§2.2 大 O、Ω、Θ、小 o

2.2.1 大 O:上界

定义(实用版): $T(n) = O(f(n))$ 表示存在常数 c 与 n_0 ,使得对所有 $n \geq n_0$,都有 $T(n) \leq c \cdot f(n)$ 。直白地说:当 n 足够大时, $T(n)$ 的增长速度不超过 $f(n)$ 的某个常数倍。例如 $T(n) = 3n^2 + 10n + 7$,取 $c = 4$ 、 n_0 足够大即可证明 $T(n) = O(n^2)$ ——因为 $10n + 7$ 相对 $3n^2$ 微不足道。

所以大 O 会“吞掉”低阶项与常数: $O(3n^2 + 10n + 7) = O(n^2)$, $O(2n) = O(n)$ 。这也是它的局限:两个同为 $O(n)$ 的算法可能实际相差 10 倍。

2.2.2 Ω 与 Θ:下界与紧界

$\Omega(f(n))$ 是对偶记号: $T(n) \geq c \cdot f(n)$ 对所有足够大的 n 成立,表示“增长不低于 $f(n)$ ”。而 $\Theta(f(n))$ 表示“恰好同阶”:既 $O(f(n))$ 又 $\Omega(f(n))$ 。工程上最常用的是大 O(上界保证“最坏不会更差”)与 Θ(精确刻画);当说“这个算法是 $\Theta(n \log n)$ ”时,意思是上下界都压紧了。

2.2.3 小 o 与三种常见误用

小 $o(f(n))$ 表示“增长速度严格低于 $f(n)$ ”(大 O 允许相等,小 o 不允许): $n = o(n^2)$ 成立,但 $n \neq o(n)$ 。它常用于比较两个上界谁更“松”。

🚀 记号说明(本章约定,全书沿用)

正文中的 $O(n)$ 读作"大 O n"; n^2 表示 n 的平方; $\log_2 n$ 表示以 2 为底的对数, 不写底数时 $\log n$ 默认以 2 为底(常数倍差异被大 O 吞掉, 所以底数不重要, 但推导过程要写清楚); $\ln$ 表示自然对数。求和写成 $\sum_{i=1}^n i = n(n+1)/2$ 。代码里一律使用语言语法(`%`、`>>`、`pow`), 不使用数学记号。

⚠ 误区 1: 把常数当成增长阶

写 $O(2n)$ 、 $O(100n \log n)$ 、 $O(n+100)$ 都是错误的: 大 O 定义明确吞掉常数, 应写 $O(n)$ 、 $O(n \log n)$ 、 $O(n)$ 。"常数不重要"仅在分析增长阶时成立; 实际工程选型时, 两个 $O(n)$ 算法的常数差距可能决定生死。

⚠ 误区 2: 混淆最坏情形与记号本身

"快排是 $O(n^2)$ 的"——这句话必须补全: 快排最坏情形是 $O(n^2)$, 平均情形是 $O(n \log n)$ 。大 O 描述的是某个约定情形(最坏/平均/最好)下的增长阶, 不是算法本身的属性。今后每章的复杂度速查表都会区分"平均"与"最坏"两列。

§2.3 增长阶排序与常见函数

2.3.1 七个等级

- $O(1)$ 常数: 按下标访问列表元素、dict 查找(平均)
- $O(\log n)$ 对数: 二分查找—— n 翻倍只多一步
- $O(n)$ 线性: 一趟遍历、线性搜索
- $O(n \log n)$ 线性对数: 归并排序、快排(平均)、堆排序
- $O(n^2)$ 平方: 二重循环、插入/冒泡排序、朴素矩阵乘法
- $O(n^3)$ 立方: 三重循环、Floyd 全源最短路
- $O(2^n)$ 指数: 穷举、朴素递归斐波那契——现实规模下不可用

2.3.2 数值表: 让直觉落地

$f(n)$	$n=10$	$n=100$	$n=10^3$	$n=10^4$	$n=10^5$	$n=10^6$
$\log_2 n$	3.3	6.6	10	13	17	20
n	10	100	10^3	10^4	10^5	10^6
$n \log_2 n$	33	664	10^4	1.3×10^5	1.7×10^6	2×10^7
n^2	100	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	1024	约 10^{30}	约 10^{301}	约 10^{3010}	约 10^{30103}	约 10^{301030}

读这张表的要领: 从 $n=10^3$ 到 10^6 , 规模扩大 1000 倍—— n 增长 1000 倍, n^2 增长 100 万倍, 2^n 则直接冲出宇宙。同时注意 $\log_2 n$ 一行: 规模增长 1000 倍, 它只从 10 长到 20。对数函数是"几乎不增长"的函数, 凡是对数级的算法都值得信赖。Python 的大整数能精确算出这张表里 2^{100} 的每一位数字——不妨在 REPL 里亲自验证一下。

⚠ 误区 3: 把 $2n$ 与 n^2 混为一谈

2^n 与 n^2 只差一个“位置”, 命运完全不同: $n = 64$ 时, $n^2 = 4096$, 而 $2^n \approx 1.8 \times 10^{19}$ 。判断口诀: 底数是 n 的多项式, 指数是 n 的才是指数级。 1.1^n 也是指数级(底数略大于 1 仍是灾难), n^{100} 再大也只是多项式。

§2.4 运行时间计算规则

2.4.1 三条基本规则

- **顺序相加**: 两个语句依次执行, 总时间取两者之和, 大 O 取最大者: $O(n) + O(n^2) = O(n^2)$
- **嵌套相乘**: 循环体内嵌套循环, 总时间相乘
- **分支取最大**: `if/else` 取两个分支中较慢的一个

2.4.2 循环与求和记号

循环本质上是一个求和: 第 i 次迭代的工作量 $w(i)$, 总工作量 $= \sum_i w(i)$ 。例:

示例 2-2 嵌套循环的求和推导

```
# 推导: 内层循环执行  $n - i$  次
# 总次数  $= \sum_{i=0}^{n-1} (n - i) = n + (n-1) + \dots + 1 = n(n+1)/2$ 
# 所以是  $O(n^2)$ , 而不是“感觉上的  $n^2/2$ ”——常数  $1/2$  被大  $O$  吞掉
# 对照: C 的写法 for (i = 0; i < n; i++) 完全一致, 结论不变
cnt = 0
for i in range(n):
    for j in range(i, n):
        cnt += 1
```

三个最常用的求和公式(本章及全书反复使用):

- $\sum_{i=1}^n i = n(n+1)/2$ (算术级数, $O(n^2)$)
- $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ (几何级数, $O(2^n)$)
- $\sum_{i=1}^n 1/i \approx \ln n$ (调和级数, $O(\log n)$)

2.4.3 递归: 递推式入门

递归代码的复杂度由**递推式**描述: $T(n) = 2T(n/2) + n$ 表示“把规模 n 的问题分成两个 $n/2$ 的子问题, 再花 $O(n)$ 合并”。逐层展开: 第 1 层代价 n , 第 2 层 $2 \cdot (n/2) = n$, 第 3 层 $4 \cdot (n/4) = n \dots$ 共 $\log_2 n$ 层, 每层 n , 总计 $O(n \log n)$ 。§2.5 的算法三正是这个递推式, §2.6 的三个对数算法则是 $T(n) = T(n/2) + O(1)$ 的特例——每层只进一个分支, 共 $\log_2 n$ 层, 每层常数代价。

★ 重点: 两种最经典的递推式

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分查找); $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (归并/分治)。全书的对数与线性对数复杂度, 九成来自这两个式子。

§2.5 最大子序列和:四种算法的递进

2.5.1 问题描述

给定整数序列 $a_1, a_2, \dots, a_n$ (可含负数), 求其**连续子序列的最大和**; 全为负数时约定答案为 0 (空子序列)。例如 $[-2, 11, -4, 13, -5, -2]$ 的最大子序列是 $[11, -4, 13]$, 和为 20。这是 Weiss 教材的经典例题: 同一个问题, 四种算法, 复杂度相差 100 万倍。

2.5.2 算法一: $O(n^3)$ 暴力枚举

最直接的思路: 枚举所有起点 i 与终点 j , 再累加 $a[i..j]$ 。双重循环枚举起终点, 内层再用 `sum()` 花 $O(n)$ 求和, 总代价 $\sum \sum (j-i+1) \approx n^3/6$, 即 $O(n^3)$ 。

示例 2-3 算法一: 枚举 + `sum()`, $O(n^3)$

```
def max_sub_sum1(a):
    n = len(a)
    max_sum = 0
    for i in range(n):           # 起点
        for j in range(i, n):    # 终点
            this_sum = sum(a[i:j+1]) # 切片求和, 本身就是  $O(j-i+1)$ 
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum
```

2.5.3 算法二: $O(n^2)$ 复用中间和

观察: 固定起点 i , 当终点从 j 推进到 $j+1$ 时, 新和 = 旧和 + $a[j+1]$, 不需要重算整个区间。去掉切片求和, 复杂度降到 $\sum 1 = n(n+1)/2 = O(n^2)$ 。

示例 2-4 算法二: 增量更新, $O(n^2)$

```
def max_sub_sum2(a):
    max_sum = 0
    for i in range(len(a)):
        this_sum = 0
        for j in range(i, len(a)):
            this_sum += a[j]      # 增量更新, 不再重算
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum
```

2.5.4 算法三: $O(n \log n)$ 分治

分治三步: 把数组分成左右两半, 递归求左半最大和与右半最大和, 再求**跨中界的最大和**(从中界向左延伸的最大后缀和 + 向右延伸的最大前缀和), 三者取最大。递推式正是 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ 。递归深度只有 $\log_2 n$, 即使 $n = 10^6$ 也不到 20 层, Python 默认 1000 层的递归上限完全够用。

示例 2-5 算法三: 分治, $O(n \log n)$

```
def max_sum_rec(a, left, right):
    if left == right:                # 基准情形:单个元素
        return max(a[left], 0)

    center = (left + right) // 2
    max_left = max_sum_rec(a, left, center)    # 左半
    max_right = max_sum_rec(a, center + 1, right) # 右半

    max_left_border = left_border = 0    # 跨中界:向左延伸
    for i in range(center, left - 1, -1):
        left_border += a[i]
        max_left_border = max(max_left_border, left_border)
    max_right_border = right_border = 0    # 向右延伸
    for i in range(center + 1, right + 1):
        right_border += a[i]
        max_right_border = max(max_right_border, right_border)

    return max(max_left, max_right,
               max_left_border + max_right_border)

def max_sub_sum3(a):
    return max_sum_rec(a, 0, len(a) - 1)
```

2.5.5 算法四:O(n) 在线算法

关键观察:和为负的前缀不可能是最优解的起点——负前缀只会拖累后面。于是扫描一遍:累加当前和 `this_sum`,一旦 `this_sum` 变成负数就清零重来;途中记录最大值。

示例 2-6 算法四:在线算法, $O(n)$

```
def max_sub_sum4(a):
    max_sum = this_sum = 0
    for x in a:
        this_sum += x
        if this_sum > max_sum:
            max_sum = this_sum
        elif this_sum < 0:
            this_sum = 0    # 负前缀弃之,从下一个元素重新开始
    return max_sum
```

例题 2-1 手工走一遍算法四

输入 `[-2, 11, -4, 13, -5, -2]`:`this_sum` 依次为 `-2→0(清零)`、`11`、`7`、`20`、`15`、`13`; `max_sum` 记录峰值为 `20`。答案 `20`,与算法一致。**注意:**清零动作丢弃的是“负前缀”而不是负数本身——`a[i]` 单独为负但前缀仍正时不会清零。

★ 重点:四种算法问题对照

算法	思想	复杂度	$n=10^6$ 时(1 微秒/步)
一	暴力枚举	$O(n^3)$	约 317 年
二	增量更新	$O(n^2)$	约 11.6 天
三	分治	$O(n \log n)$	约 20 秒
四	在线扫描	$O(n)$	约 1 秒

§2.6 对数级复杂度的三种来源

2.6.1 二分查找

在有序数组中查找 x : 每次把候选区间减半。区间长度 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$, 共 $\log_2 n$ 步, 即递推式 $T(n) = T(n/2) + O(1)$ 。

示例 2-7 二分查找, $O(\log n)$

```
# 在升序列表 a 中查找 x, 返回下标; 找不到返回 -1
def binary_search(a, x):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] < x:
            low = mid + 1
        elif a[mid] > x:
            high = mid - 1
        else:
            return mid
    return -1
```

示例 2-8 标准库写法: bisect 一行搞定

```
import bisect
a = [1, 3, 5, 7, 9]
i = bisect.bisect_left(a, 5)    # 返回 2: 第一个 >= 5 的位置
j = bisect.bisect_left(a, 6)    # 返回 3: 6 应插在 7 之前
bisect.insort_left(a, 6)        # 保持有序地插入: 1 3 5 6 7 9
k = bisect.bisect_right(a, 5)   # 返回 3: 第一个 > 5 的位置
n = bisect.bisect_right(a, 5) - i # 等于 5 的元素个数: 1
print(i, j, k, n, a)           # 2 3 3 1 [1, 3, 5, 6, 7, 9]
```

C 的手写二分查找	Python 的标准库
<pre>int bin(int a[], int n, int x)</pre> 手写约 12 行	<code>bisect_left / bisect_right / insort</code>
边界条件 (<code><=</code> 还是 <code><</code>) 极易写错成死循环	边界由标准库保证, 写错直接抛异常暴露
数组要单独传长度 n	<code>bisect.bisect_left(a, x)</code> 一行, 列表自含长度

手写 vs 标准库:学习时手写一遍理解"减半"骨架;工程中直接用 `bisect` ——它用 C 实现,而且避免了边界条件(`<=>` 写错一个就死循环)的错误。同一思想,C++ 标准库对应 `std::binary_search` 与 `std::lower_bound`。

2.6.2 快速幂

计算 $a^n \bmod m$:朴素做法乘 n 次是 $O(n)$;利用 $a^n = (a^2)^{n/2}$ 每次把指数减半, $O(\log n)$ 。这是 C 里你可能用循环写过的算法,与二分查找共享"减半"骨架。

示例 2-9 快速幂, $O(\log n)$

```
MOD = 10**9 + 7

def pow_mod(a, n):
    """计算 (a^n) % MOD, n >= 0. """
    result = 1
    while n > 0:
        if n & 1:                # 当前二进制位为 1 则乘入
            result = result * a % MOD
        a = a * a % MOD          # a 平方:对应指数减半
        n >>= 1                  # n 右移一位 = n//2
    return result

# Python 内置 pow 三参数版直接就是快速幂:
print(pow_mod(2, 10), pow(2, 10, MOD)) # 1024 1024
```

2.6.3 欧几里得算法

求最大公因数 $\gcd(a, b)$:迭代执行 $(a, b) \rightarrow (b, a \% b)$ 直到 $b = 0$ 。结论:每两步至少把较小的数减半(证明略),所以迭代次数 $O(\log n)$ 。

示例 2-10 欧几里得算法, $O(\log n)$

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b        # 元组打包/解包,C 里要写三行
    return a

import math
print(gcd(48, 36), math.gcd(48, 36)) # 12 12,标准库同样有现成的
print(gcd(1234567, 891011))          # 大数同样两步减半,依旧很快
```

★ 重点:log 从哪来?

三个算法的共同骨架: 每一步把"问题规模"至少减半。二分查找减半区间,快速幂减半指数,欧几里得每两步减半数。凡是见到"减半""倍增"字样的算法,先猜 $O(\log n)$ 再验证,十有八九正确。

§2.7 实测验证:让理论落地

2.7.1 计时方法要点

- n 要足够大:太小则计时噪声淹没规律(§2.7.2 用 $n \geq 1000$)
- 重复取平均或取中位数,避免一次性的调度抖动

- 关闭不必要的后台程序,尽量独占 CPU
- **警惕 Python 特有偏差:**内置函数(如 `sum`、`min`)是 C 实现的,比手写循环快一个数量级——比较"算法"时必须让双方使用同样风格的代码,否则比的是实现语言而非算法

C 的计时要点	Python 的计时要点
<code>clock()</code> 测 CPU 时间,受调度影响	<code>perf_counter()</code> 测墙上时间,单调不回拨
除以 <code>CLOCKS_PER_SEC</code> 得到秒,容易忘记	返回值就是秒(浮点),直接相减
手动 <code>volatile</code> 防优化,易漏	解释器不做死代码消除,但结果同样要输出

2.7.2 n 翻倍实验(Python 实测参考值)

对随机数组运行四种算法,每次 n 翻倍,观察时间比值:

n	算法一	算法二	算法三	算法四
1000	24 ms	21 ms	1.6 ms	0.06 ms
2000	186 ms	84 ms	3.4 ms	0.12 ms
4000	≈1.5 s(推算)	336 ms	7.2 ms	0.24 ms
8000	≈12 s(推算)	1.35 s	15 ms	0.48 ms

读出规律:n 翻倍,算法一时间 ×8(吻合 n^3),算法二 ×4(吻合 n^2),算法三、四约 ×2(吻合 $n \log n$ 与 n)。比值比绝对值可靠——你的机器数值不同,但比值规律必然一致。与 C++ 版对比:绝对时间慢 30–100 倍,但比值规律完全相同——增长阶是语言无关的。

工程建议:常数因子的真实意义

大 O 相同不代表同样快:同为 $O(n \log n)$,快排的常数通常只有堆排的一半;同为 $O(n)$,遍历 `list` 比遍历手写链表快数倍(缓存友好,第3章细讲)。在 Python 里这条原则加倍重要:`sum(a[i:j+1])` 是 C 实现的,写起来只有一行,但它是 $O(j-i+1)$ ——常数小救不了复杂度差。工程选型时:先用大 O 淘汰指数/高次算法,再用常数因子与实测在候选者中择优。

误区 4:小规模下拿 O 记号赌博

$n = 20$ 时, $O(n^2)$ 的插入排序往往比 $O(n \log n)$ 的快排更快——常数小。大 O 的结论只在“n 足够大”时成立(定义里的 n_0 正是为此)。工程上排序库普遍采用“大区间快排 + 小区间插入排序”的混合策略(第7章),就是这条原则的产物。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
按值查找	无序列表	$O(n)$	$O(n)$
二分查找	有序列表	$O(\log n)$	$O(\log n)$
求 a^n	朴素连乘	$O(n)$	$O(n)$
求 a^n	快速幂 / 内置 pow(三参数)	$O(\log n)$	$O(\log n)$
最大公因数	欧几里得算法 / math.gcd	$O(\log n)$	$O(\log n)$
最大子序列和	算法一(暴力)	$O(n^3)$	$O(n^3)$
最大子序列和	算法二(增量)	$O(n^2)$	$O(n^2)$
最大子序列和	算法三(分治)	$O(n \log n)$	$O(n \log n)$
最大子序列和	算法四(在线)	$O(n)$	$O(n)$
循环计数	单层循环	$O(n)$	$O(n)$
循环计数	双重循环	$O(n^2)$	$O(n^2)$
递归	$T(n) = T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$
递归	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n \log n)$
增长阶	常数 / 对数 / 线性 / 线性对数 / 平方 / 立方 / 指数	见 §2.3.1 排序	
增长阶排序	常数 $< \log n < n < n \log n < n^2 < n^3 < 2^n$	全书唯一需要背诵的排序	

最小必会清单

- 能独立说出大 O 、 Ω 、 Θ 、小 o 的定义与区别,并各举一例
- 能背诵增长阶排序,并给每个等级配一个真实算法例子
- 能独立推导单层循环、嵌套循环、if/else 的复杂度
- 能独立写出四种最大子序列和算法,并说出各自复杂度与推导
- 能独立写出二分查找、快速幂、gcd 的代码,并解释为什么是 $O(\log n)$
- 能写出 $T(n) = T(n/2) + O(1)$ 与 $T(n) = 2T(n/2) + O(n)$ 的解
- 能解释"最坏情形"与"平均情形"的区别,并正确表述快排的复杂度
- 能独立用 perf_counter 做 n 翻倍实验,判断实验比值与理论增长阶是否吻合

自测题

Q 1. $O(2n)$ 与 $O(n)$ 是否相等?为什么?

✅ 参考答案

相等。大 O 定义允许任意常数倍:取 $c = 2$ 即有 $2n \leq 2 \cdot n, 2n = O(n)$ 。反过来 $n \leq 1 \cdot 2n$, 所以两者互为对方的 O , 增长阶相同。写 $O(2n)$ 是不规范的。

Q 2. 某算法运行时间 $T(n) = 3n^2 + 10n + 7$, 它的 O 、 Ω 、 Θ 分别可以写什么?

✓ 参考答案

$O(n^2)$ (也允许 $O(n^3)$ 等更松的上界,但不规范); $\Omega(n^2)$ (也允许 $\Omega(n)$ 等更松的下界); $\Theta(n^2)$ 是唯一"紧"的写法。

Q 3. 为什么说算法三(分治)比算法二(增量)快?写出算法三的递推式并解释每层代价。

✓ 参考答案

$T(n) = 2T(n/2) + n$:把数组分为两半递归($2T(n/2)$),再花 $O(n)$ 求跨中界最大和。展开后第 k 层有 2^k 个规模 $n/2^k$ 的子问题,每层总代价都是 n ,共 $\log_2 n$ 层,总计 $n \log_2 n$ 。 $O(n \log n)$ 在 $n = 10^6$ 时约 2×10^7 步,而 $O(n^2)$ 是 10^{12} 步,差 5 万倍。

Q 4. 二分查找每次把区间减半,为什么复杂度是 $O(\log n)$ 而不是 $O(n/2)$?

✓ 参考答案

区间长度依次为 $n, n/2, n/4, \dots, 1$,问"减半多少次到 1"——这是对数: $2^k = n$,即 $k = \log_2 n$ 次。 $n/2$ 是"一次减半后的长度",不是步数;把长度当步数是常见错误。 10^6 个元素只需约 20 次比较。

Q 5. $n = 1000$ 时, $O(n^2)$ 与 $O(n \log n)$ 的步数约相差多少倍? $n = 10^6$ 呢?

✓ 参考答案

$n = 1000: 10^6 / (1000 \times 10) = 100$ 倍; $n = 10^6: 10^{12} / (10^6 \times 20) = 5 \times 10^4$ 倍。规模越大差距越大——这就是为什么"复杂度差的算法靠硬件换代救不回来"。

Q 6. Python 程序比 C++ 慢约 50 倍,那么 $O(n)$ 的 Python 算法与 $O(n \log n)$ 的 C++ 算法,数据规模多大时 Python 反而更快?

✓ 参考答案

比较 $50n$ 与 $n \log_2 n$:当 $\log_2 n > 50$,即 $n > 2^{50} \approx 10^{15}$ 时 Python 更快——现实规模基本达不到,所以"语言换效率"在增长阶面前通常无效。反过来: $O(n^2)$ 的 C++ 在大 n 下照样被 $O(n \log n)$ 的 Python 反超。

章末实验题:四种算法的增长阶实测报告

实验 2:验证最大子序列和四种算法的增长阶

实验目标:实现四种最大子序列和算法,对倍增规模计时,用实测比值验证 $O(n^3)/O(n^2)/O(n \log n)/O(n)$ 的增长阶预测。

实验要求:

- 任务 1:实现 max_sub_sum1..4 四个函数(知识点:\$2.5 四种算法)
- 任务 2:用 perf_counter 计时,输出 $n = 1000/2000/4000/8000$ 的时间表(知识点:\$2.1 计时工具)
- 任务 3:算法一在 $n \geq 4000$ 时跳过并标注"(推算)",避免无谓等待(知识点:\$2.3 指数/立方增长不可用)
- 任务 4:输出" n 翻倍时各算法时间比值",与理论值($\times 8/\times 4/\times 2/\times 2$)对比并写一段结论(知识点:\$2.7 实测验证)

实验环境:Python 3.8+,标准库即可。运行: `python lab2.py`。

第一步 写四个算法函数,用 \$2.5 的样例数组 `[-2, 11, -4, 13, -5, -2]` 验证四个函数都输出 20,再写计时主程序

第二步 机数用 `random.seed(12345) + random.randint(-1000, 1000)` 生成,保证可复现

第三步 时用 perf_counter,每次只测一次即可(时间基数大,噪声占比小);把四次调用结果累加并输出,防止解释器优化掉调用

第四步 比值分析:对算法二,若 n 翻倍时间约 $\times 4$,则吻合 n^2 ;对算法三、四约 $\times 2$;算法一只测两个规模并说明 $\times 8$ 趋势

✓ 参考答案(完整可运行程序,约 70 行,分两段)

```

# lab2.py — 最大子序列和:四种算法的增长阶实测(第一段:算法)
import random
import time

# ----- 四种算法($2.5) -----
def max_sub_sum1(a):                                #  $O(n^3)$ 
    n = len(a)
    max_sum = 0
    for i in range(n):
        for j in range(i, n):
            this_sum = sum(a[i:j+1])
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum

def max_sub_sum2(a):                                #  $O(n^2)$ 
    max_sum = 0
    for i in range(len(a)):
        this_sum = 0
        for j in range(i, len(a)):
            this_sum += a[j]
            if this_sum > max_sum:
                max_sum = this_sum
    return max_sum

def max_sum_rec(a, left, right):                    #  $O(n \log n)$  分治
    if left == right:
        return max(a[left], 0)
    center = (left + right) // 2
    max_left = max_sum_rec(a, left, center)
    max_right = max_sum_rec(a, center + 1, right)
    max_left_border = left_border = 0
    for i in range(center, left - 1, -1):
        left_border += a[i]
        max_left_border = max(max_left_border, left_border)
    max_right_border = right_border = 0
    for i in range(center + 1, right + 1):
        right_border += a[i]
        max_right_border = max(max_right_border, right_border)
    return max(max_left, max_right,
               max_left_border + max_right_border)

def max_sub_sum3(a):
    return max_sum_rec(a, 0, len(a) - 1)

def max_sub_sum4(a):                                #  $O(n)$  在线算法
    max_sum = this_sum = 0
    for x in a:
        this_sum += x
        if this_sum > max_sum:
            max_sum = this_sum
        elif this_sum < 0:
            this_sum = 0
    return max_sum

```

```
# ----- 计时($2.1) -----
```

lab2.py(第二段:计时与主程序)

```
def ms_time(func, a):
    t0 = time.perf_counter()
    result = func(a)
    return (time.perf_counter() - t0) * 1000, result

def main():
    # 正确性验证($2.5 样例)
    demo = [-2, 11, -4, 13, -5, -2]
    results = [max_sub_sum1(demo), max_sub_sum2(demo),
               max_sub_sum3(demo), max_sub_sum4(demo)]
    print(f"验证: {results} (应全为 20)")

    random.seed(12345)                # 固定种子,可复现
    guard = 0
    print("\n\t算法一\t算法二\t算法三\t算法四")
    for n in (1000, 2000, 4000, 8000):
        a = [random.randint(-1000, 1000) for _ in range(n)]
        row = [str(n)]
        if n <= 2000:
            t, r = ms_time(max_sub_sum1, a)
            row.append(f"{t:.1f}")
            guard += r
        else:
            row.append("--(推算)")        #  $O(n^3)$  在 4000 时约 1.5 秒,不实测
        for f in (max_sub_sum2, max_sub_sum3, max_sub_sum4):
            t, r = ms_time(f, a)
            row.append(f"{t:.3f}")
            guard += r
        print("\t".join(row))
    print(f"(防优化累加值: {guard % 1000})")
    print("结论: n 翻倍时,算法一  $\times 8$ (吻合  $n^3$ ),算法二  $\times 4$ (吻合  $n^2$ ),
          "算法三/四约  $\times 2$ (吻合  $n \log n$  与  $n$ )。")

if __name__ == "__main__":
    main()
```

示例输出(机器不同数值不同,比值规律一致):

```
验证: [20, 20, 20, 20] (应全为 20)
n      算法一   算法二   算法三   算法四
1000   24.0     21.00    1.600    0.060
2000  186.0    84.00    3.400    0.120
4000  --(推算) 336.0    7.200    0.240
8000  --(推算) 1350.0   15.00    0.480
(防优化累加值: 321)
结论: n 翻倍时,算法一  $\times 8$ (吻合  $n^3$ ),算法二  $\times 4$ (吻合  $n^2$ ),算法三/四约  $\times 2$ (吻合  $n \log n$  与  $n$ )。
```

设计说明:① 四个算法共用 `ms_time` 计时函数,把"被测函数"当参数传入——Python 函数是一等对象,这正是第10章高阶函数思想的雏形,也对应 C 的函数指针;② `guard` 累加所有返回值并输出,防止解释器/编译器优化掉调用;③

算法一在 $n \geq 4000$ 时按 §2.3 的数值表推算($\times 8$ 每翻倍),避免浪费时间在早已被理论淘汰的算法上;④ 种子固定为 12345,任何机器复现本实验都得到相同的数组。

下一章预告:第3章 表、栈和队列——三种最基础的数据结构及其数组/链式实现。第3章起,每章末尾的复杂度速查表将直接使用本章的记号;最大子序列和的分治骨架也将在第7章排序中再次登场。